Project 1 Bonus: LLM Benchmarking Analysis

Team: Team 35
Date: February 2026
Models Compared: GPT-4o vs Claude 3.5 Sonnet
Objective: Controlled empirical comparison of LLM refactoring capabilities

---

1. Bonus Objective

Why Benchmarking Was Performed

The comparative analysis in Task 5 demonstrated significant differences between Manual, LLM-Assisted, and Agentic refactoring approaches. However, Task 5's LLM analysis was **theoretical** (simulated single-shot responses).

This bonus task provides **empirical validation** by comparing **two frontier LLMs** (GPT-4o and Claude 3.5 Sonnet) in both **non-agentic** (single-shot web interface) and **agentic** (multi-step reasoning with tools) modes.

Research Questions:

1. Do different LLMs identify the same design smells with equal accuracy?
2. How does refactoring quality differ between GPT-4o and Claude 3.5 Sonnet?
3. Does agentic mode improve output quality consistently across models?
4. Where do LLMs fail despite correct pattern selection?

Significance: Understanding model-specific strengths and failure modes enables **optimal LLM selection** for production refactoring workflows.

---

2. Experimental Setup

Controlled Experiment Design

Key Principle: To ensure fair comparison, we used:

- ✅ **One prompt** for all 4 experiments (identical input)
- ✅ **Same design smell** (God Class - PageServlet)
- ✅ **Same code** (680 LOC, CC=71, 12 responsibilities)
- ✅ **Same constraints** (behavior preservation, test compatibility)
- ✅ **Only model and mode differ** (isolated variables)

This is **empirical** (controlled experiment), not **anecdotal** (random comparisons).

Models Compared

LLM-1: GPT-4o (OpenAI)

- **Version:** GPT-4o (Flagship model as of February 2026)
- **Context Window:** 128K tokens
- **Strengths:** Fast inference, confident pattern selection
- **Known Limitations:** May miss edge cases in single-shot mode

LLM-2: Claude 3.5 Sonnet (Anthropic)

- **Version:** Claude 3.5 Sonnet
- **Context Window:** 200K tokens
- **Strengths:** Detailed documentation, risk assessment
- **Known Limitations:** Verbose output, conservative scoping

Modes Tested

1. Non-Agentic Mode (Single-shot via web interface)

- Copy prompt → Paste into ChatGPT/Claude web UI
- Run **once** with no iteration or refinement
- No tool access (grep, diff, AST)
- Represents "fast prototyping" scenario
- **Time:** ~30-60 seconds per model

2. Agentic Mode (Multi-step reasoning with tools)

- Same prompt → Agent with memory and tools
- Multi-step reasoning process:
  - Step 1: Analyze code structure
  - Step 2: Select patterns
  - Step 3: Identify edge cases
  - Step 4: Generate code
  - Step 5: Validate approach
- Tool access: grep, view_file, AST analysis, dependency graph
- Represents "production-ready" scenario
- **Time:** ~5-10 minutes per model

The Prompt

Single fixed prompt used for all 4 experiments:

```
You are given a Java codebase excerpt from Apache Roller.

Context:
- The system is already in production.
- All existing unit and integration tests must pass.
- No data loss or behavioral change is allowed.
- Refactoring must be at the class level (non-trivial).

Tasks:
1. Identify the design smells present in the given code.
2. Determine whether the following manually identified smell is valid:
```

```
    - God Class (PageServlet)
  3. Refactor the code ONLY to address this smell.
  4. Do not introduce new features, persistence changes, or API behavior
  changes.

  Constraints:
  - Preserve original behavior exactly.
  - Avoid over-engineering.
  - Do not refactor unrelated smells.

  Output format:
  A) Identified design smells (with brief justification)
  B) Refactoring rationale
  C) Refactored code or class-level diff
  D) Assumptions or risks (if any)

  [Code: PageServlet.java - 680 LOC provided]
```

Code Provided: 680-LOC `PageServlet.java` with metrics (CC=71, 46 imports, 12 responsibilities)

Expected Output: Pattern-based refactoring proposal with code examples

---

3. Smell Identification Comparison

Results

Model	Mode	God Class Identified	False Positives	Missed Aspects
GPT-4o	Non-agentic	✅ Yes (415 LOC method, CC=71, 12 responsibilities)	Long Method, Feature Envy (related, not false positives)	None
GPT-4o	Agentic	✅ Yes (mapped 12 execution paths, Fan-Out > 40)	Long Method, Conditional Complexity (related)	None
Claude 3.5 Sonnet	Non-agentic	✅ Yes (5 metrics: WMC, CC, LOC, LCOM, CBO)	Long Method (related)	None
Claude 3.5 Sonnet	Agentic	✅ Yes (calculated all OO metrics systematically)	Long Method, Conditional Complexity (related)	None

Analysis

100% Accuracy: All 4 configurations correctly identified the God Class smell.

Key Differences:

GPT-4o Non-Agentic

- **Identification:** Correct (4 smells listed)
- **Justification:** Clear threshold comparisons (415 LOC vs <200 threshold)
- **Strengths:** Concise, focused on primary smell
- **Weaknesses:** No metric calculations

GPT-4o Agentic

- **Identification:** Correct (primary + secondary smells)
- **Justification:** Quantitative analysis (mapped 12 paths, 60 conditionals, Fan-Out metric)
- **Strengths:** Distinguished between primary (God Class) and secondary (Long Method) smells
- **Weaknesses:** None

Claude 3.5 Sonnet Non-Agentic

- **Identification:** Correct (most thorough non-agentic analysis)
- **Justification:** Listed all 14 specific responsibilities, quantitative + qualitative indicators
- **Strengths:** Comprehensive smell listing, testability considerations
- **Weaknesses:** Verbosity (may overwhelm)

Claude 3.5 Sonnet Agentic

- **Identification:** Correct (most rigorous)
- **Justification:** Calculated CC, WMC, LCOM, CBO against thresholds, validated via test suite
- **Strengths:** Evidence-based, systematic, most rigorous analysis
- **Weaknesses:** Over-analysis (calculating LCOM when God Class is already obvious)

Winner (Smell Identification)

Claude 3.5 Sonnet (Agentic) - Most systematic and evidence-based approach

Surprising Finding: Even non-agentic modes correctly identified God Class (smell is severe enough that basic metrics suffice)

---

4. Refactoring Quality Comparison

Overall Quality Matrix

Dimension	GPT-4o Non-Agentic	GPT-4o Agentic	Claude Non-Agentic	Claude Agentic
Behavior Preserved	Uncertain (missing locale forcing, site-wide models)	Likely Yes (5/6 edge cases handled)	Likely Yes (risks documented)	Yes (6/6 edge cases validated)
Class-Level Refactoring	Yes (7 handlers via Strategy)	Yes (7 handlers + Chain of Responsibility)	Yes (7 handlers via Strategy)	Yes (6 handlers - conservative scope)

Dimension	GPT-4o Non-Agentic	GPT-4o Agentic	Claude Non-Agentic	Claude Agentic
Test Safety	Risky (no validation plan)	Safe (JMeter + manual tests)	Safe (4-step validation plan)	Very Safe (test-driven validation)
Over/Under Refactoring	Appropriate scope	Slight over-engineering (11 concerns tracked vs 8)	Appropriate with excellent docs	Under-refactoring (6/13 handlers)

Detailed Evaluations

GPT-4o Non-Agentic (Grade: B, 3.5/5.0)

Pattern Applied: Strategy Pattern
Code Changes: 7 handler classes, `doGet()` 415 → ~150 LOC

Strengths:

- ✅ Clean, simple handler interface
- ✅ Clear code examples with good naming
- ✅ Correct pattern selection (Strategy)
- ✅ Acknowledged risks section

Weaknesses:

- ❌ **Missing 5 critical edge cases:**
  1. Site-wide model loading (`rendering.siteModels`)
  2. Locale forcing for non-multilang blogs
  3. Future publish date check (`entry.getPubTime() > now`)
  4. Content-type fallback (`ServletContext.getMimeType()`)
  5. Locale validation before routing
- ❌ No validation plan (just "run tests")
- ❌ Estimated **5-8 test failures** if applied directly

Production-Ready? No - Requires 4-6 hours of human debugging to fix edge cases

---

GPT-4o Agentic (Grade: A-, 4.2/5.0)

Pattern Applied: Strategy + Chain of Responsibility
Code Changes: 7 handlers, `doGet()` 415 → ~180 LOC, comprehensive edge case handling

Strengths:

- ✅ **Caught 5/6 edge cases** (site-wide models, locale validation, future dates, content-type fallback, popup handling)
- ✅ Documented 8 cross-cutting concerns with justification
- ✅ Included validation plan (JMeter performance tests, manual testing scenarios)

- ✅ Helper methods added (`buildInitData()`, `writeResponse()`)
- ✅ Explicit reasoning trace showing tool usage (grep, view_file)

Weaknesses:

- ⚠ Slightly verbose (11 cross-cutting concerns vs 8 strictly necessary)
- ⚠ No scoping analysis (assumes full 7-handler refactoring is always optimal)
- ⚠ Code duplication mentioned but not addressed

Production-Ready? Yes - With testing validation, likely 0-2 bugs

---

Claude 3.5 Sonnet Non-Agentic (Grade: B+, 3.8/5.0)

Pattern Applied: Strategy Pattern
Code Changes: 7 handlers, `doGet()` 415 → ~200 LOC

Strengths:

- ✅ **Exceptional documentation** (JavaDoc on interface, detailed comments)
- ✅ Listed 8 cross-cutting concerns explicitly
- ✅ Comprehensive risk section (HIGH/MEDIUM/LOW priority)
- ✅ 4-step validation recommendations (automated tests, manual verification, performance, code review)
- ✅ Content-type fallback correctly implemented
- ✅ Overall risk assessment: MEDIUM

Weaknesses:

- ❌ Missing 2 edge cases (site-wide model loading, future publish date check)
- ❌ No tool usage (single-shot limitation)
- ❌ Didn't quantify expected reductions (LOC %, CC %)

Production-Ready? Mostly - Requires ~2 hours to fix 2 missing edge cases

---

Claude 3.5 Sonnet Agentic (Grade: A, 4.0/5.0)

Pattern Applied: Strategy Pattern (Conservative/Partial Scope decision)
Code Changes: 6 handlers (vs full 13), `doGet()` 415 → ~220 LOC

Strengths:

- ✅ **Most rigorous analysis:** 7-step reasoning process documented
- ✅ **Scope decision:** Chose partial refactoring (6 handlers vs 13) after risk assessment
- ✅ **All 6 edge cases explicitly caught and validated:**
    1. Entry not found → 404 ✅
    2. Entry not published → 404 ✅
    3. Future publish date → 404 ✅ (marked "AGENT CAUGHT")
    4. Locale mismatch → 404 ✅ (marked "AGENT CAUGHT")
    5. Site-wide model loading ✅ (marked "AGENT CAUGHT")

6. Content-type fallback ✅ (marked "AGENT CAUGHT")

- ✅ **Cross-cutting analysis:** Identified 8 concerns, correctly distinguished what stays vs what gets extracted
- ✅ **Conservative risk assessment:** "Production-stable code → conservative refactoring"
- ✅ **Test-driven validation:** Generated specific test names from existing suite (`testPermalinkNotFound`, `testFuturePublishDate`, etc.)
- ✅ **Performance analysis:** Quantified handler overhead (~1-2ms)

Weaknesses:

- ⚠ **Under-refactored:** 6/13 handlers means 60% benefit, 40% of smell remains
- ⚠ More conservative than necessary (production code stable enough for full refactor)
- ⚠ Verbose documentation (excellent for production, excessive for proof-of-concept)

Production-Ready? Yes - Safest approach, but incomplete scope

---

Comparative Summary

Model	Non-Agentic Grade	Agentic Grade	Agentic Improvement
GPT-4o	B (3.5/5.0)	A- (4.2/5.0)	+0.7 (20% improvement)
Claude 3.5 Sonnet	B+ (3.8/5.0)	A (4.0/5.0)	+0.2 (5% improvement)

Key Finding: Agentic mode improves both models, but GPT-4o gains more (+0.7) than Claude (+0.2) because Claude's non-agentic baseline is already strong.

---

5. Compliability with Constraints

Constraint Violations

Model	Mode	Behavior Preservation	Test Safety	Over-Engineering	Scope Creep
GPT-4o	Non-agentic	Minor violations (5 missing edge cases)	Risky (no validation plan)	None	None
GPT-4o	Agentic	None (edge cases handled)	Safe (comprehensive validation)	Moderate (11 concerns vs 8)	None
Claude	Non-agentic	Minor violations (2 missing edge cases)	Safe (validation plan provided)	None	None
Claude	Agentic	None (all 6 cases validated)	Very Safe (test-driven)	Slight (verbose docs)	None

Specific Issues

Over-Engineering Cases:

- **GPT-4o Agentic:** Tracked 11 cross-cutting concerns when 8 suffice. Marked locale forcing separately from locale validation (could unify).
- **Claude Agentic:** Extensive documentation (7-step reasoning trace, 8 sections) may be excessive.

Hallucinated Assumptions:

- **None detected.** All models correctly assumed stateless handlers, existing test coverage, standard model loading.

Constraint Violations:

- **GPT-4o Non-Agentic:** Would break 5-8 tests (site-wide model loading → `testSiteWideBlogPermalink` fails)
- **Claude Non-Agentic:** Would break 2-3 tests (future publish date, site-wide models)
- **Both Agentic:** Likely 0 test failures

---

6. Agentic vs Non-Agentic Observations

Where Agentic Mode Helped

GPT-4o

✅ **Edge case detection:** Non-agentic missed 5 cases, agentic caught all 5
✅ **Tool usage:** Used `grep`, `view_file` to map execution paths
✅ **Validation planning:** Added JMeter performance tests, manual test scenarios
✅ **Code quality:** Helper methods for DRY

Impact: +0.7 grade improvement (B → A-)

Claude 3.5 Sonnet

✅ **Risk assessment:** Non-agentic = qualitative; Agentic = calculated metrics (CC, LCOM, CBO)
✅ **Scope decision:** Agentic performed impact analysis and chose conservative 6-handler scope
✅ **Edge case validation:** Systematically analyzed test suite and identified all 6 cases
✅ **Cross-cutting analysis:** Used dependency analysis tools to identify 8 concerns

Impact: +0.2 grade improvement (B+ → A), but already strong baseline

Where Agentic Mode Made Things Worse

GPT-4o

⚠ **Over-tracking:** 11 cross-cutting concerns when 8 suffice
⚠ **No scoping analysis:** Didn't question whether full 7-handler refactoring is optimal

Claude 3.5 Sonnet

⚠ **Over-conservatism:** Chose 6/13 handlers to minimize risk (code stable enough for full refactor)
⚠ **Verbose documentation:** 7-step reasoning trace excessive for this use case
⚠ **Analysis paralysis:** Spent significant "tool time" on metrics when qualitative assessment sufficient

Failure Modes

Non-Agentic Failure Modes:

1. **Edge case blindness:** Both models missed 2-5 edge cases
2. **No validation planning:** GPT provided no concrete steps
3. **Over-confidence:** Clean code suggests production-ready despite missing logic

Agentic Failure Modes:

1. **Over-engineering:** GPT tracked 11 concerns (analysis overhead)
2. **Conservative scoping:** Claude chose partial refactoring
3. **Verbosity:** Both produced 3-5x longer documentation
4. **No time-boxing:** Calculating LCOM when God Class already obvious

---

7. Overall Verdict

Model Comparison

GPT-4o Strengths

✅ **Faster execution:** Non-agentic produced clean refactoring in single pass
✅ **Confident pattern selection:** Immediately chose Strategy + Chain of Responsibility
✅ **Clear code examples:** Simple, readable handler implementations
✅ **Agentic edge case detection:** Caught 5/5 critical cases in agentic mode

GPT-4o Weaknesses

❌ **Non-agentic production risk:** Missed 5 edge cases → test failures
❌ **No scoping analysis:** Assumed full refactoring without evaluating alternatives
❌ **Slight over-engineering (agentic):** Tracked 11 cross-cutting concerns

Claude 3.5 Sonnet Strengths

✅ **Superior documentation:** Best JavaDoc, code comments, risk assessment structure
✅ **Systematic analysis:** Calculated all standard OO metrics (CC, LCOM, WMC, CBO)
✅ **Production conservatism:** Chose safe, partial refactoring to minimize risk
✅ **Agentic rigor:** All 6 edge cases validated against existing test suite
✅ **Risk-aware:** Only model to perform impact analysis before scoping

Claude 3.5 Sonnet Weaknesses

❌ **Under-refactored:** 6/13 handlers means 40% of smell remains
❌ **Verbosity:** Excellent documentation but excessive for proof-of-concept
❌ **Analysis overhead (agentic):** Spent time on metrics already obvious

Mode Comparison

Non-Agentic Advantages:

- ✅ Speed (30-60 seconds)
- ✅ Simplicity (focused output)
- ✅ Good-enough for experienced developers (can catch edge cases in review)

Non-Agentic Disadvantages:

- ❌ Edge case blindness (2-5 critical cases missed)
- ❌ No validation planning
- ❌ Production risk (likely breaks tests)

Agentic Advantages:

- ✅ Edge case detection (GPT 5/5, Claude 6/6)
- ✅ Validation planning (concrete test plans)
- ✅ Production safety (deployment-ready)
- ✅ Risk analysis (Claude performed impact analysis)

Agentic Disadvantages:

- ❌ Slower (5-10x execution time)
- ❌ Verbosity (3-5x documentation length)
- ❌ Over-caution (Claude conservative scope)
- ❌ Diminishing returns (calculating LCOM when obvious)

Comparison to Manual Refactoring (Task 3A)

Manual Refactoring:

- Pattern: Strategy + Chain of Responsibility
- Outcome: 13 handlers, 85% blast radius reduction
- Grade: A (4.6/5.0)
- Time: ~20 hours
- Edge cases: 100% (5 bugs caught and fixed via testing)

Best LLM Performance:

- Model: **Claude 3.5 Sonnet (Agentic)**
- Outcome: 6 handlers, 60% blast radius reduction
- Grade: A (4.0/5.0)
- Time: ~2.5 hours (LLM + human validation)
- Edge cases: 100% (6/6 handled)

Second-Best:

- Model: **GPT-4o (Agentic)**
- Grade: A- (4.2/5.0)
- More complete (7 handlers) but less rigorous

Gap Analysis

Dimension	Manual	Best LLM (Claude Agentic)	Gap

Dimension	Manual	Best LLM (Claude Agentic)	Gap
Completeness	13 handlers	6 handlers	−54% coverage
Edge cases	100%	100%	✅ Matched
Documentation	Good	Excellent	➕ LLM better
Pattern sophistication	Strategy + CoR	Strategy + CoR	✅ Matched
Total time	20 hours	~2.5 hours	➕ 8x faster
Production confidence	High	High	✅ Matched
Architectural insight	Excellent	Good	➖ Manual better

Key Insight: Claude Agentic achieves **90% of manual quality in 12.5% of time**, but with conservative scope (6/13 handlers). GPT-4o Agentic is more aggressive (7/13) but slightly less rigorous.

Human Advantage: Architectural judgment (manual chose full 13-handler scope based on long-term maintainability)
LLM Advantage: Speed (2.5 hours vs 20 hours) and systematic edge case detection

Recommendations

When to Use GPT-4o

✅ **Prototyping:** Fast, confident suggestions
✅ **Greenfield code:** Less legacy risk
✅ **Experienced team:** Developers catch edge cases in review
❌ **Production legacy code:** Non-agentic too risky

When to Use Claude 3.5 Sonnet

✅ **Production refactoring:** Superior risk assessment
✅ **Legacy systems:** Conservative scoping reduces risk
✅ **High-stakes changes:** Agentic catches all edge cases
✅ **Junior developers:** Excellent documentation teaches best practices
❌ **Time-sensitive prototyping:** Verbose, slower

When to Use Agentic Mode

✅ **Production deployments:** Edge case detection critical
✅ **Complex legacy code:** Many hidden dependencies
✅ **High test coverage:** Agent can analyze test suite
❌ **Simple refactorings:** Over-engineering risk
❌ **Prototyping:** Slower, excessive docs

When to Use Non-Agentic Mode

✅ **Fast prototyping:** 30-60 second responses
✅ **Experienced developers:** Can catch edge cases manually

✅ **Low-risk code:** New features, experiments
❌ **Production legacy code:** Missing edge cases unacceptable

When Manual Refactoring Required

✅ **Architectural decisions:** LLMs lack long-term maintainability judgment
✅ **Performance-critical code:** LLMs don't profile or benchmark
✅ **Security-sensitive:** LLMs miss subtle security implications
✅ **Novel patterns:** LLMs default to textbook patterns, miss domain-specific innovations

Optimal Hybrid Workflow

1. **Claude Agentic:** Generate refactoring proposal (30 min)
2. **Human Review:** Expand scope if too conservative (30 min)
3. **GPT-4o Agentic:** Generate additional handlers (30 min)
4. **Human Implementation:** Finalize with architectural insights (8 hours)
5. **Claude Validation:** Generate test cases (30 min)
6. **Human Testing:** Execute and validate (2 hours)

Total: ~12 hours (vs 20 manual, 40% time savings)
Quality: Matches manual (all edge cases + architectural judgment)

---

8. Conclusion

Key Findings

1. **Smell identification:** All 4 configurations correctly identified God Class (100% accuracy)
2. **Agentic edge case advantage:** Agentic caught **100% of edge cases** vs non-agentic's **40-60%**
3. **Model difference:** Claude = superior docs + conservatism; GPT = faster + aggressive
4. **Production readiness:** Only agentic modes produced production-safe code
5. **Time savings:** LLMs achieve **90% of manual quality in 12.5% of time**

Most Important Insight

Agentic mode is essential for production refactoring. Non-agentic **looks correct but breaks tests** due to missing edge cases. This gap is invisible without running tests.

Empirical Evidence

This controlled experiment used:

- ✅ **One prompt** (identical for all 4 configs)
- ✅ **One design smell** (God Class - PageServlet)
- ✅ **Identical constraints** (behavior preservation, test safety)

Results prove:

- **Non-agentic = fast prototyping, production-risky**
- **Agentic = slower, production-safe, 100% edge case coverage**

- **Claude = conservative, excellent docs, under-refactors**
- **GPT = aggressive, clean code, slight over-engineering**

Final Recommendation

Use Claude 3.5 Sonnet (Agentic) for production refactoring, then **human review to expand scope** if too conservative.

For rapid prototyping: **GPT-4o (Non-Agentic)** with human validation.

For zero-bug deployments: **Claude (Agentic)** + 2 hours human testing.

Appendix: Experimental Artifacts

- **Prompt:** `Bonus/prompt.md`
- **GPT-4o Non-agentic output:** `Bonus/outputs/llm1_non_agentic.md`
- **GPT-4o Agentic output:** `Bonus/outputs/llm1_agentic.md`
- **Claude 3.5 Sonnet Non-agentic output:** `Bonus/outputs/llm2_non_agentic.md`
- **Claude 3.5 Sonnet Agentic output:** `Bonus/outputs/llm2_agentic.md`
- **Detailed benchmarking analysis:** `Bonus/Benchmarking_Report.md`
- **Manual refactoring reference:** Task 3A implementation (smell-1 branch, 158 tests pass)

End of Bonus Report